Multi-agent STARS forecasting and SEAN-JAX social navigation

Project report: code changes, experiments, results and assumptions · work of 21–23 October 2026, compiled 27 October 2026

Repository: `michaelmunje/stars_steps_extension` (private), branch `scale-and-sim` · working copy `/scratch/09865/mmunje/stars-forecasting` on TACC Horizon

Summary

Over three days on TACC Horizon (GB200 GPUs) we did four things.

- **Extended STARS into a multi-agent pedestrian forecaster.** STARS was a self-supervised social representation model; it now forecasts every pedestrian in a scene jointly, with a CVAE decoder for multiple futures and closed-loop rollouts for simulation.
- **Built a scalable training system around it.** It has multi-GPU DDP, bf16, on-GPU graph construction, YAML dataset mixes and Optuna search, and reaches 174k pedestrians/s on one 4-GPU node. We trained on STEPS and on six, then seven, datasets.
- **Wrote SEAN-JAX, a new batched 2-D social-navigation simulator, from scratch.** It is the Phase 1–2 successor to the Unity/ROS-based SEAN 2.0. Its pedestrians can be driven by the STARS forecaster itself.
- **Tuned a 13-parameter social navigation controller with CMA-ES** inside that forecaster-driven crowd, then tested it on 270 held-out scenes.

Outcome	Headline (test split)
Best STEPS forecaster (cvae_kl01_finetune)	Agent ADE / FDE 0.080 / 0.126 m (best of 20), against 0.276 / 0.518 m for constant velocity. For moving people: 0.336 / 0.588 against 0.894 / 1.771.
Pooled forecaster (cvae_all_lr3e-4_seed42)	2–3.5× lower error than constant velocity on all six datasets. Better than or level with a per-dataset model on each of them (ETH +41%, UCY +23%).
Seven-dataset forecaster with ATC (new, not previously reported)	ATC ADE 0.296 m against 0.310 m zero-shot (−4.5%). The other six datasets change by at most ±2%.
Navigation policy v4 (cmaes_crowd_v4)	In the STARS crowd: 99% success, 0.7% collisions, 2.6% of time within 0.45 m of a person, and 100% success on 30 recorded STEPS scenes. Goal-seeking reaches 38% success; the untuned controller 83%.
Attempts to beat v4	None succeeded: cost-term variants, memory, wait-then-go plans, a learned MLP policy with a safety shield, and contextual parameter networks.

What SEAN-JAX is, in one paragraph. SEAN-JAX contains **no code copied from SEAN 2.0**. SEAN 2.0 is a C#/Unity application with a ROS 1 bridge, and none of it can run on this cluster. We read its source to inventory its features, then wrote a new batched simulator in PyTorch (not JAX, despite the name, which you chose). It reproduces SEAN 2.0's behaviours and definitions: the social-force agent structure, the social-situation taxonomy, and the intimate (0.45 m) and personal (1.2 m) zone metrics. The ORCA pedestrian model is a line-by-line algorithmic port of the RVO2 C++ library (Apache-2.0), checked against RVO2's Python binding to about 1e-7 m/s. The only third-party runtime libraries are PyTorch, NumPy, matplotlib and pycma. The data-driven crowd and the predictor reuse our own STARS forecaster code, which in turn wraps the unmodified upstream STARS encoder. Section 5 has the module-by-module breakdown.

1 Scope, timeline and environment

When (2026)	Work
21 Oct, morning	Environment: dotfiles; GitHub CLI login; SSH key; GLM-5.3-Flash vLLM recipe (ntllm) prepared; uv Python environment with CUDA torch for aarch64.
21 Oct, evening	Read the STARS model's inputs and outputs. Implemented the multi-agent STARS extension (graph, encoder, decoders, rollout, metrics, tests) and validated it on CPU.
21–22 Oct, night	Moved to STEPS v2 and deleted the legacy STEPS data. Shared the datasets on TACC and pushed the private repo. Scaled training (DDP, bf16, batching) and benchmarked on one node. Started Optuna. Wrote SEAN-JAX Phase 1, then ORCA (Phase 2b) and the STARS predictor.
22 Oct	30-minute autonomous progress loop. Forecaster ablations, the KL-weight finding and all-dataset models. Built the STARS-driven crowd and tuned crowd policies v1 → v4. Pooled LR fix. Published the latest-policy and latest-forecaster pages, with GIFs.
23 Oct	Controller structure experiments (decisions 81–90). Learned and contextual policies (91–92). ATC as a seventh dataset (93); its jobs finished after the session ended and are first reported here.

Compute and environment

- TACC Horizon: Rocky Linux 10 on aarch64, 4× GB200 (189 GB each) per node, Slurm partition `debug`, allocation CDA26011. No sudo; the login node has one CPU and no GPU.
- The Python environment is managed with uv. PyPI's aarch64 torch wheels are CPU-only, so a `[tool.uv.sources]` entry with a `platform_machine` marker pulls torch and torchvision from the PyTorch cu130 index.
- Builds must use `CC=gcc CXX=g++`. The default NVHPC compiler (`nvc`) breaks C extensions (`ncls`, `pyrvo`) and `torch.compile`.
- Compute nodes have no internet. Kernels (flashinfer cubins, 6.2 GB) and weights are pre-downloaded on the login node.

2 Codebases touched

Location	What changed	State
`~/.dotfiles`, `~/.zshrc`, `~/.bashrc`	nathantsoi/dotfiles installed in user-local mode; git identity set.	Backups in `~/dotfiles-preinstall-backup-20261021/`.
`~/.ssh/authorized_keys`	Horizon ed25519 key appended.	Previous file kept as `authorized_keys.bak`.
`$SCRATCH/ntllm` (GLM recipe)	`glm53_flash.slurm`: correct HERE path, `unset TMUX TMUX_PANE` (an inherited `$TMUX` overrides `TMUX_TMPDIR`, tmux.c:544), launch via `respawn-window`.	Uncommitted. The server never reached ready; the last attempt ended in a node failure.
`$SCRATCH/trajectory_forecasting` (main checkout)	pyproject: cu130 index plus 11 missing dependencies; requirements and uv.lock regenerated; .venv built (560 tests pass).	Uncommitted. CLAUDE.md and 51 manifests still have stale /home/mike paths.
`$SCRATCH/stars-forecasting` → GitHub `stars_steps_extension`	All project work: the multi-agent STARS forecaster, the training system, SEAN-JAX, configs, Slurm scripts, docs, tests and figures.	Pushed. `main` has the scaling work to d72eb22; `scale-and-sim` has the rest (116 commits) up to 7c47579.
`$SCRATCH/shared/datasets`	Datasets moved here and made world-readable (o+rX), with symlinks from both checkouts. STEPS = indoor v2 only; all legacy STEPS data, including outdoor, was deleted (you have backups).	Live.
`~/src/tmux-3.7c`	None. It is the tmux build installed to ~/.local.	—

> **Security note.** A GitHub personal access token was pasted in plain text into the chat on 21 October and used for *gh auth login*. If it has not already been revoked and rotated, do that.

3 Data

All datasets go through the repository's existing data pipeline into a shared scene format: positions [T, A, 2], with NaN where an agent is absent, plus validity masks, sampled at 2.5 Hz (dt = 0.4 s). Each window has 8 observed steps (3.2 s) and 12 predicted steps (4.8 s). Train, validation and test splits are grouped by recording, so no recording appears in two splits.

Dataset	Role	Notes
STEPS (indoor v2)	Primary; the SEAN-JAX crowd and recorded scenes	198 indoor atrium recordings. Train: 101,032 samples, 76,675 unique windows. Test: 12,500 windows. About 90% of pedestrians move slower than 0.2 m/s (mean speed about 0.17 m/s).
PTDPES	Pooled training	Large; similar in scale to STEPS.
Madras, Grand Central, UCY, ETH	Pooled training	ETH has no validation windows under the grouped split (two recordings), so it is tested but not used for model selection.
ATC (two-day subset)	Seventh dataset (decision 93)	The 22 May recording is split 90/10 into train and val along its timeline; the 29 September recording is the whole test split. The index has 8.08M heavily overlapping windows, capped by a seeded draw to 80k train, 10k val and 50k test.

4 Multi-agent STARS forecaster

4.1 What came from STARS and what is new

Upstream STARS (`ut-amrl/stars` at 13d840c, pinned as a submodule in `references/stars`) is a graph VAE. It encodes SE(2) relative-pose edge sequences between agents into a social representation, resampled to a 40-step grid, and was probed for a single target agent. The original single-target probe (`baselines.train_stars`) is unchanged and stays as a baseline.

Component	Origin	File
Type embedding, edge TemporalEdgeTransformer, TransformerConv stack, to_mu / to_logvar	**Upstream STARS, unmodified** (imported through baselines.stars.upstream_model_module). Positional embeddings are resized from 40 to 8 steps.	`stars_ma_model.py`
8-dim SE(2) edge features	**Upstream definition.** One change: before an agent's first movement, its heading now takes that movement's direction instead of the scene x axis, which makes the features SE(2)-invariant (tested).	`stars_ma_graph.py`
Node motion branch: 7 local-frame features (position, velocity, speed, relative heading), a second temporal transformer, and a bias-free projection	**New**, using the same mechanism STARS uses to inject SEAN robot features.	`stars_ma_model.py`
kNN / radius topology (16 neighbours within 10 m), on-GPU batched graph building	**New.** STARS used fully connected graphs; that is still available as a flag.	`stars_ma_graph.py`, `stars_ma_batch.py`
Trajectory decoder: social TransformerConv round, then a GRU emitting local displacements; deterministic or CVAE (learned prior and posterior per agent)	**New.** The CVAE latent is separate from STARS's representation latent.	`stars_ma_model.py`
Closed-loop rollout (predict → execute k steps → rebuild graph)	**New**	`stars_ma_model.py`
Metrics: best-of-K target / agent / joint ADE / FDE, moving vs stationary, interacting vs isolated, density bins, collisions, diversity, motion distributions	**New**	`stars_ma_metrics.py`
Training, evaluation, HPO and reporting scripts	**New**	`train_stars_ma.py`, `evaluate_stars_ma.py`, `hpo_stars_ma.py`, …

Training proceeds in phases. Phase 1 is STARS's own self-supervised VAE objective plus node-motion reconstruction. Phase 2 trains the forecaster with the encoder frozen, fine-tuned or from scratch. Phase 3 is the CVAE. Phase 4 adds a multi-step closed-loop loss through rebuilt graphs. The model is permutation-equivariant and size-agnostic, with a reserved robot node type (type 0).

4.2 Training system and throughput

- Windows are cached per split in a GPU-resident `WindowStore` and graphs are built in batches on the device. Batches are sized in pedestrians (`--max-nodes`), not windows.
- Multi-GPU runs use DDP via torchrun. bf16 autocast is the default, with the GRU recurrence and graph construction kept in fp32. LR scales with the square root of the global batch, with warm-up and cosine decay.
- Temporal transformers run in chunks of up to 32,768 sequences, because fused attention refuses more than 65,535. Activation checkpointing is optional.
- Datasets are listed in YAML configs (steps, all, all7, ptdpes, steps_plus_ptdpes) with per-dataset weights and window caps. Checkpoints are selected on the macro mean of per-dataset validation agent ADE.
- Optuna uses journal-file storage, one worker per GPU and a median pruner.

Mode (job 65466, one GB200 node)	Pedestrians / batch	Pedestrians / s	Peak memory
Phase 1 SSL, bf16	16,384	50k	57 GB
Phase 1 SSL, fp32	16,384	33k	92 GB
Phase 1, fully connected graphs	16,384	33k	140 GB
Phase 1, 4 GPUs DDP	16,384 × 4	174k (87% scaling)	57 GB
CVAE forecaster, 1 GPU / 4 GPUs	16,384	48k / 168k	59 GB
CVAE + 4-step rollout loss	4,096	10.6k	59 GB
Any mode	65,536	out of memory	—

4.3 Forecasting results on STEPS

Model (STEPS test, 12,500 windows)	Agent ADE / FDE	Moving ADE / FDE	Joint ADE
Constant velocity	0.276 / 0.518	0.894 / 1.771	0.279
Deterministic decoder, fine-tuned phase-1 encoder	0.115 / 0.205	0.559 / 1.107	0.119
CVAE, KL 1.0, fine-tuned (collapsed latent)	0.110 / 0.195	0.532 / 1.049	—
CVAE, KL 0.1, fine-tuned (cvae_kl01_finetune), the reference	**0.080 / 0.126**	**0.336 / 0.588**	**0.097**
CVAE, KL 0.1, from scratch with search settings	0.081 / 0.129	0.342 / 0.600	0.097
Best Optuna trial 53, full length (2 seeds)	0.079 / 0.125	0.333 / 0.587	0.097

Best of 20 samples for the CVAE rows, in metres. Moving means a true mean future speed above 0.2 m/s.

Findings

- **The KL weight is the setting that matters.** At 1.0 the CVAE latent collapses: the 20 samples end about 9 mm apart. At 0.1 they spread about 0.16 m and error falls 30–36%. Two Optuna searches (475 scratch trials, 23 complete and 452 pruned; 213 SSL fine-tuning trials) attribute 51–58% of the variance to it (fANOVA), and 2–4% to batch size. Both searches converged; the top eight scratch trials are within 0.0005 m, below seed noise.
- **Seed noise** is about ±0.0007 m of validation ADE.
- **Pretraining helps slightly** once KL is right: fine-tuned beats from scratch, which beats frozen. Pretraining on all six datasets adds nothing on STEPS.
- **Closed-loop (multi-step) training does not help.** Rollouts are already stable without it, and exec-1 closed-loop fine-tunes were unstable and over budget.

- **Most accurate is not most realistic.** Trial 53 is marginally more accurate, but its simulated pedestrians walk 25% too slowly. cvae_kl01_finetune's 20 s rollouts move at 0.115 m/s, matching smoothed real tracks, so it drives the simulator. Its rollouts are smoother than reality: acceleration 0.03 against 0.12 m/s², and most of the real acceleration is tracking jitter.

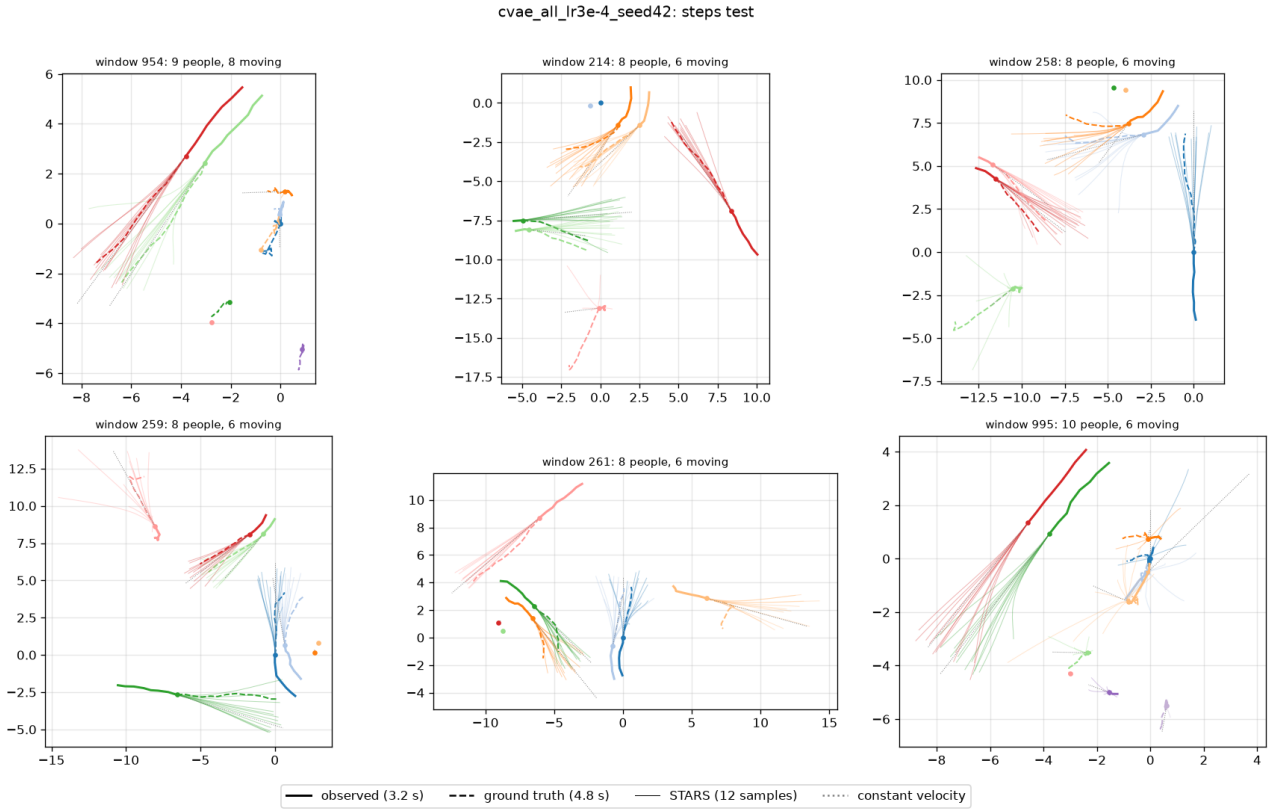


Figure 1. Pooled model on STEPS test windows: observed 3.2 s (solid), ground truth (dashed), 12 STARS samples (thin) and constant velocity (dotted).

4.4 Pooled training across datasets

A single model trained on the six-dataset mix first proved seed-sensitive at the LR tuned on STEPS (9.6e-4): on average it was worse than per-dataset models on the large datasets (decisions 84–85). Lowering the pooled LR to 3e-4 fixed this (decision 86). Across four runs the pooled model is then better than or level with each dataset's own model everywhere: STEPS within ±1%; PTDPES, Madras and Grand Central 1–3% better; UCY 22–24% and ETH 40–42% better.

Dataset	Constant velocity	6-dataset reference (ATC zero-shot)	7-dataset, seed 42	7-dataset, seed 1
STEPS	0.276 / 0.518	0.079 / 0.124	0.078 / 0.123	0.079 / 0.124
PTDPES	0.963 / 1.800	0.282 / 0.409	0.281 / 0.403	0.284 / 0.413
Madras	0.627 / 1.180	0.209 / 0.313	0.214 / 0.324	0.213 / 0.324
Grand Central	0.703 / 1.427	0.243 / 0.403	0.240 / 0.395	0.244 / 0.404
UCY	0.397 / 0.891	0.199 / 0.395	0.195 / 0.386	0.194 / 0.379
ETH	0.727 / 1.428	0.232 / 0.378	0.232 / 0.366	0.229 / 0.365
ATC	0.973 / 1.821	0.310 / 0.429	0.296 / 0.416	0.297 / 0.415

Agent ADE / FDE in metres, best of 20, test splits. The ATC jobs (67749, 67750) finished on 23 October after the session ended; these numbers are first reported here.

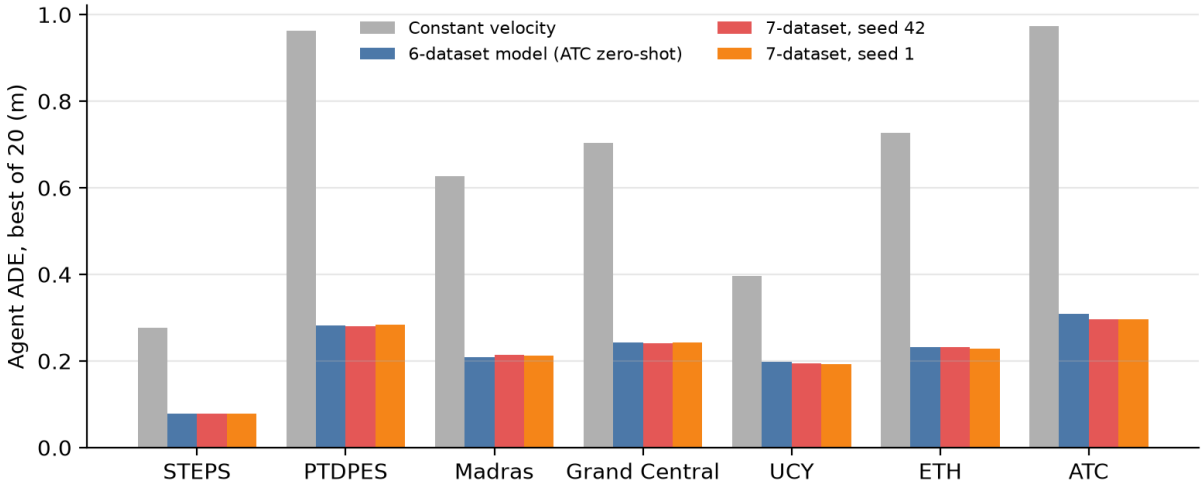


Figure 2. Per-dataset test ADE with ATC added. Training on ATC gives a 4.5% gain on ATC and leaves the six original datasets within about ±2% (Madras 0.209 → 0.213; ETH and UCY slightly better).

- **ATC.** The six-dataset model already generalises zero-shot to ATC (0.310 m against 0.973 m for constant velocity). Adding a capped ATC to training lowers this to 0.296–0.297 m. Both seeds agree to within 0.001 m on ATC.
- **Self-feeding.** When the model feeds back its own predictions every step, error rises little on STEPS but 14–20% on the other datasets.
- SSL pretraining plus fine-tuning is 0.002–0.010 m worse than from scratch on every pooled dataset.

5 SEAN-JAX: what it is and where its code comes from

5.1 Starting point

You supplied a 13-section specification for a "SEAN 3.0" simulator. In outline:

- Phase 1: a lightweight batched 2-D backend with replay, social-force, ORCA and reactive-replay pedestrians, a parameterised social navigation policy, CMA-ES tuning, metrics, visualisation and tests.
- Phase 2: learned pedestrians.
- Phase 3: Isaac Sim / Isaac Lab.
- Phase 4: ROS 2.

The spec allowed PyTorch with JAX optional. You asked for the name **seanjax**; the package was renamed from `sean3`. It is implemented entirely in PyTorch and no JAX port is planned (decision 16).

Before writing code we surveyed the original projects. Shallow clones of `yale-sean/social_sim_unity`, `social_sim_ros`, `sim_ws` and `snape/RVO2` are in `$SCRATCH/.cache/sean_survey/`. SEAN 2.0 turned out to be:

- **Engine:** Unity 2020.3 (C#), with the ROS-TCP-Connector and the URDF importer.
- **ROS side:** ROS 1 Noetic in an Ubuntu 20.04 CUDA container.
- **Robots:** seven URDF robots (Jackal, Fetch, TurtleBot3, Warthog, Kuri, P3DX, Unitree A1).
- **Pedestrians:** social-force agents (`SFAgent.cs`) following Unity NavMesh paths. Its ORCA agent (`SEAN/ORCA/Agent.cs`) is only a stub that throws NotImplemented.
- **Situations and metrics:** a rule-based online social-situation classifier, and metrics published as a ROS `TrialInfo` message.

None of this runs on Horizon: no Unity, no ROS 1, Rocky Linux rather than Ubuntu, aarch64, and no display. So SEAN-JAX is a **clean-room reimplementation of SEAN 2.0's behaviour**, not a fork.

5.2 Provenance, module by module

The categories used below are:

- **New**: designed and written in this project.
- **Re-implemented from SEAN 2.0**: behaviour or definitions taken from SEAN's C# source and written fresh in batched PyTorch.
- **Ported**: an algorithm translated from another codebase's source.
- **Library**: an external package called at runtime.

Module (src/seanjax/…)	Lines	Provenance	Details
core/state.py	99	New	Batched ScenarioBatch, RobotState, PedestrianState and a backend-independent Observation. Every environment is one row of a tensor batch.
core/scenarios.py	258	New; situation set from SEAN 2.0	Ten seeded, randomised scenario kinds, with disjoint seed ranges per split. The kinds cover SEAN's cross_path, down_path and group situations: crossing, head_on, following, overtaking, stationary_group and group_toward. narrow_passage, dense_flow, empty and recorded are new.
core/geometry.py	95	New	Distances to circles and wall segments, and clearances.
core/rollout.py	67	New	Recorded episodes and the robot-absent counterfactual run (same seeds and intents, robot removed), which disruption metrics compare against.

Module (src/seanjax/…)	Lines	Provenance	Details
core/metrics.py	157	Mixed: SEAN 2.0 definitions plus new	From SEAN 2.0's Metrics.cs: intimate (< 0.45 m) and personal (< 1.2 m) zone violations, collisions, path length, minimum distance, robot-at-fault counting. Re-implemented as per-step distance thresholds rather than Unity trigger events. New: success, timeout, time to goal, path efficiency, loops, stops and emergency stops, smoothness, jerk, counterfactual disruption, disruption_local and robot_influence. SEAN 2.0 declared several of these but never implemented them.
environments/lightweight/env.py	153	New	Differential-drive disc robot with Jackal-like limits (radius 0.3 m, −0.3 to 1.0 m/s, 1.5 rad/s), acceleration limits and exact arcs. Also collisions, success and timeout; dt = 0.1 s.
pedestrians/replay.py	17	New; behaviour from SEAN 2.0 playback	Replays recorded or scripted positions exactly. SEAN played ETH/Zara CSVs; we use a documented CSV schema and STEPS.
pedestrians/social_force.py	135	Re-implemented (Helbing–Molnar; structure from SEAN 2.0)	Same structure as SFAgent.cs: goal relaxation, exponential pedestrian, robot and wall repulsion, and a sideways step. Our own constants are in m/s² (A = 2.1, B = 0.3, robot 5.0 / 0.5) plus anisotropy λ = 0.35. SEAN's A = 500, B = 0.16 are in Unity force units and were not copied.
pedestrians/reactive_replay.py	68	New	Tracks the reference trajectory, avoids using the social forces, and returns to its path. An undisturbed walker reproduces its reference exactly (tested).
pedestrians/orca.py	415	Ported from RVO2 (C++, Apache-2.0)	Batched port of Agent::computeNewVelocity, linearProgram1/2/3 and RVO2's two-vertex obstacle lines. Our additions: the robot as a non-reciprocal moving obstacle, static discs, and reference-tracking preferred velocities.
pedestrians/stars_crowd.py	173	New; uses our STARS forecaster	The forecaster moves the crowd, with the robot as one more graph node. See 5.4.
policies/social_navigation.py	300	New	Trajectory-scoring controller over a 6 × 11 grid of (v, w) commands, with a hard safety constraint and 13 tunable parameters. It replaces SEAN's move_base baseline.
policies/goal_seeking.py, base.py	120	New	Pedestrian-blind baseline, a parameter-spec system and the predictor interface.
policies/learned.py, contextual.py	320	New	MLP steering policy with a safety shield (decision 91), and an MLP that adjusts v4's settings per step (decision 92). Both are experimental.
predictors/stars.py	93	New; uses our STARS forecaster	Joint forecasts of all visible pedestrians, 4.8 s ahead, resampled to the planner's grid.
optimization/cmaes.py	153	New wrapper; library pycma (cma 4.5.0)	Ask/tell loop, warm starts, training-scene refresh, validation selection, checkpoint and resume.
optimization/evaluator.py	111	New	Evaluates a whole population × scenario set as one batched simulation; configurable objective.
datasets/trajectory_loader.py	220	New	CSV schema and STEPS converter; recorded scenes are the busiest 30 s window, split by recording.
visualization/plots.py	261	New; library matplotlib	Rollout PNGs and GIFs showing the plan, forecasts and status line; convergence plots; policy comparisons.
seanjax/scripts, configs, tests	≈ 790 + 33 configs	New	run_sim, train_cmaes, evaluate, compare_policies, visualize_rollout, diagnose_waiting, train_learned, publish_latest_policy. 30 tests in test_seanjax.py and test_seanjax_orca.py.

Line counts are from `wc -l` *at commit 7c47579. In total there are about 5,500 lines of SEAN-JAX Python, including scripts.*

What we did not take from SEAN 2.0

SEAN 2.0 feature	Status in SEAN-JAX
Unity engine, rendering, scenes (Lab, Warehouse, Outdoor, Hotel, Zara)	Replaced by the batched 2-D backend. Maps are wall segments and polygons; there is no occupancy-map import.
ROS 1 topics (/social_sim/metrics, agents, situations, cmd_vel, laser, camera, TF)	Not ported. Policies see only a backend-independent Observation so they can sit behind a ROS 2 node later (Phase 4; RoboStack provides ROS 2 Jazzy for aarch64 without sudo).
Seven URDF robots	One differential-drive disc with Jackal-like limits. URDF robots would come with Isaac (Phase 3).
NavMesh global paths for pedestrians	Pedestrians follow scripted or recorded reference trajectories instead.
Rule-based online situation classifier (2.4 m, ±15°)	Not ported. Situations are encoded as scenario kinds instead.
Cameras, LiDAR	Not supported in the lightweight backend.
ORCA (a stub in SEAN 2.0)	Implemented for real, as the RVO2 port.

5.3 How a simulation step works

- **Batching.** Every tensor has a leading environment dimension B. A CMA-ES generation (32 candidates × 72 training scenes) or a 270-scene test runs as a single batched rollout, split by max_envs = 4096. On one GB200, 768 environments of 30 s episodes take about 3 s with simple pedestrians.
- **Per step (dt = 0.1 s).** The policy sees pedestrians within 8 m and returns a (v, w) command. The robot tracks the command within its acceleration limits along an exact arc. Pedestrians then step with the robot visible (or absent, in the counterfactual). Contacts are resolved so pedestrians never interpenetrate the robot, which makes every collision the robot's fault, as in SEAN 2.0's counting.
- **Episode end.** A robot that reaches its goal leaves the scene. One that collides or times out stays as a stopped obstacle, so an early crash cannot look less disruptive than a careful pass.
- **Controller.** Each (v, w) command is rolled out over the prediction horizon against predicted pedestrians (constant velocity or STARS). Its score combines goal progress, predicted collisions, personal-space intrusion, disruption of moving pedestrians' paths, obstacle clearance, speed, turning and smoothness. Commands that come within the safety margin within the safety horizon are discarded; if none remain, the robot makes an emergency stop. Retreat, speed-fade and slow-reverse terms were added after tuned policies orbited pedestrians (decision 43).

5.4 The STARS-driven crowd

`stars_crowd` turns the forecaster into the crowd simulator:

- Each pedestrian follows its reference for 3.2 s to build history. After that, all model-driven pedestrians of all environments are forecast jointly every 0.4 s, and each moves along the first forecast step.
- CVAE intents follow a seeded AR(1) process (correlation 0.8 per 0.4 s). Holding a single draw made pedestrians spiral.
- The robot is one more person in the graph, so pedestrians react to it as they react to each other. Neither STEPS nor any other dataset contains a robot.
- Moves that would cross a wall slide along it, because the forecaster has no map.

Counterfactual displacement is chaotic in a learned crowd. Tiny forecast differences amplify over 30 s, so pedestrians who never came near the robot still end up elsewhere. The objective therefore uses **robot_influence** (the robot-attributable change in forecasts) instead (decisions 49–52).

5.5 Validation

The ORCA port was compared against `pyrvo` 0.4.3, RVO2's Python binding, built from source with gcc in a separate venv outside the repository; it is not a project dependency. "One step" feeds RVO2's state into our model at each step. "Closed loop" runs the two simulators independently.

Scene (dt 0.1 s)	One-step max |Δv| (m/s)	Closed-loop max |Δv|	Arrivals (RVO2 / ours)
Two agents head-on, offset	1e-7	2e-7	2 / 2
Four-agent swap, jittered	3e-7	4e-5	4 / 4
Eight-agent circle, jittered	1e-4	0.20 (transient)	8 / 8
Random 20 agents	6e-7	0.029	20 / 20
Dense ring, 24 agents	2.3e-4	1.66 (chaotic divergence)	24 / 24, same minimum gap
Six agents with walls	7e-7	3e-4	3 / 3
Random 20 agents with walls	6e-6	0.36	16 / 16

Exactly symmetric scenes deadlock identically in both simulators. The roughly 1e-4 one-step errors come from agents at exact contact, where float32 is ill-conditioned. Static discs are our own extension (RVO2 has none), so they are not validated against pyrvo.

The test suites cover:

- ORCA: no overlaps, speed limits, non-reciprocal robot avoidance, the counterfactual, and an LP2 solve against a brute-force optimum.
- Reactive replay: exact reference reproduction for an undisturbed walker.
- Scenarios and splits.
- The STARS crowd and predictor.
- Evaluation and visualisation, including with learned policies.

6 Navigation experiments

The objective was hand-weighted and evolved in versions:

- **v1**: success − 2·collision − disruption − 0.3·normalised time − 0.5·personal-zone rate.
- **v2** adds detour (1 − path efficiency) and command smoothness, after a reward-hacking policy circled pedestrians.
- **Crowd objectives** use robot_influence as the disruption term, comfort = time within 0.45 m with weight 1, and from v4 a charge of 0.2 per loop.

CMA-ES (pycma) setup for v4:

- Population 32, 100 generations, σ0 = 0.1, 11 optimised parameters.
- 8 training scenes per kind, refreshed every 10 generations.
- 24 validation scenes per kind, checked every 5 generations; the best checkpoint is kept.
- Test: 30 scenes per kind, 270 in all, 30 of them recorded STEPS scenes.
- Evaluation noise is about ±0.02 in the objective (bf16 on GPU).

6.1 Early stage: scripted scenes, reactive-replay pedestrians

Policy (80 scripted test scenes, objective v1)	Constant-velocity predictor	STARS predictor
Goal-seeking baseline	−0.572 (30% collisions)	—
Social controller, default	0.439 (2.5% collisions)	0.536 (0 collisions)
CMA-ES tuned v1	0.612	**0.647**

Same parameters, different predictor: the STARS forecasts help the planner (decision 37). The v1 policy had no collisions and beat the default under ORCA, social-force and replay pedestrians it was not tuned on (decision 29).

6.2 Main stage: the STARS-driven crowd

Policy (270 test scenes)	Objective	Success	Collision	< 0.45 m	Loops / ep.	Recorded success
Goal-seeking	−2.149	38%	62%	11.4%	0.00	27%
Default social	−0.196	83%	7.8%	16.4%	0.01	77%
v1 (tuned in reactive replay)	0.196	91%	0.7%	1.3%	0.09	67%
Crowd v2	0.276	96%	1.5%	5.4%	0.10	87%
Crowd v3 (wall-respecting crowd)	0.321	97%	0.4%	1.4%	0.19	77%
Crowd v4 (reference)	**0.388**	**99%**	0.7%	2.6%	**0.08**	**100%**

v4 differs from v3 in two ways: the controller's waiting costs are tuned as well, and loops are charged in the objective. All three v4 seeds push speed_scale to 2.7–5 and alignment_weight to about 0, giving a strong pull to cruising speed and no reward for pivoting toward the goal. The v4 parameters are:

goal	collision	pedestrian	disruption	obstacle	pref. speed	personal radius
0.50	5.0	27.4	0.01	1.0	1.0 m/s	0.55 m
horizon	turn penalty	smoothness	speed_scale	retreat_scale	alignment	
2.27 s	0.0	0.39	5.0 (upper bound)	1.59	0.075	

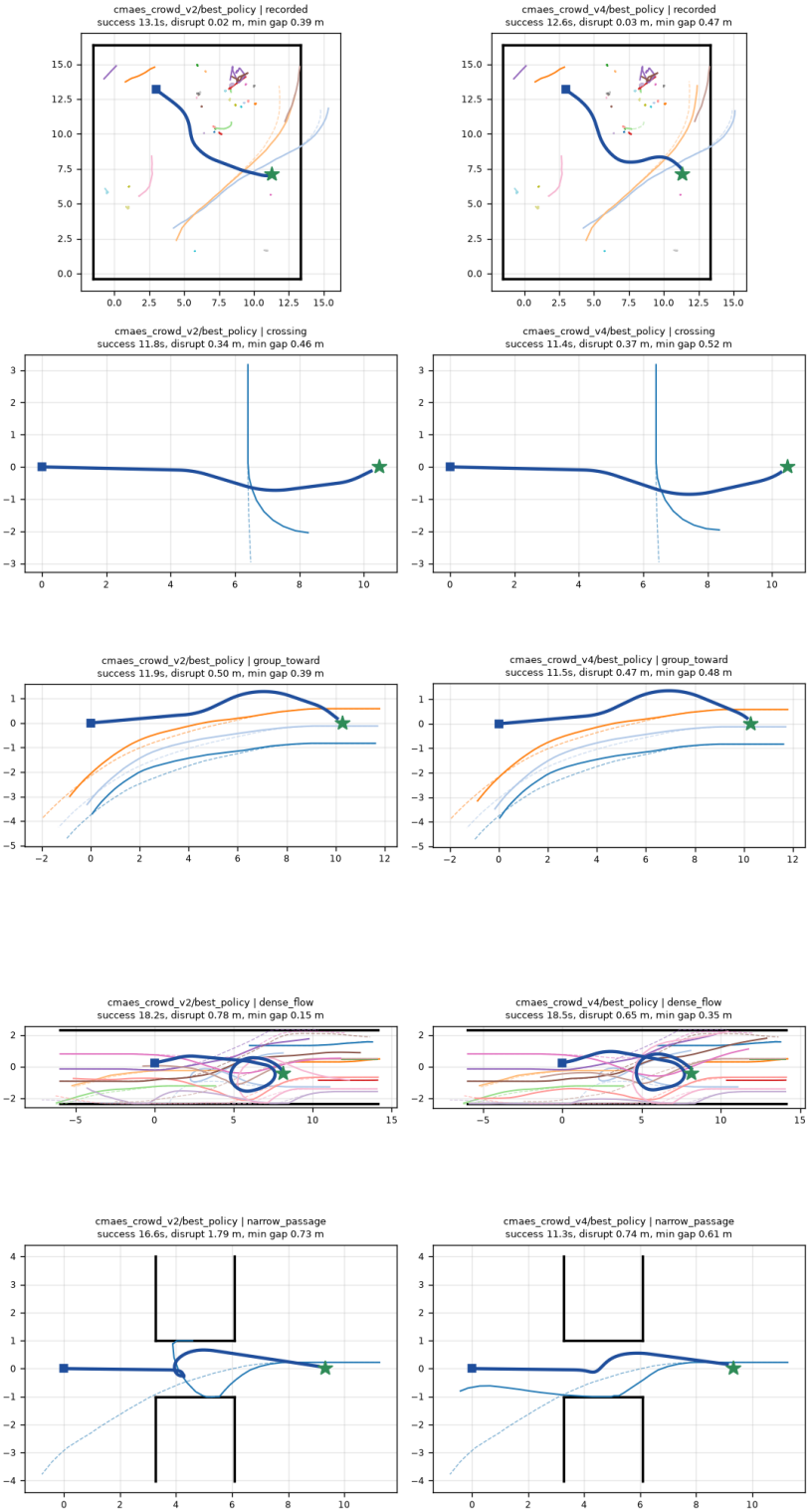


Figure 3. Crowd v2 (left) against v4 (right) on recorded, crossing, group, dense-flow and narrow-passage scenes. v4 removes the narrow-passage loop; the dense-flow loop remains.

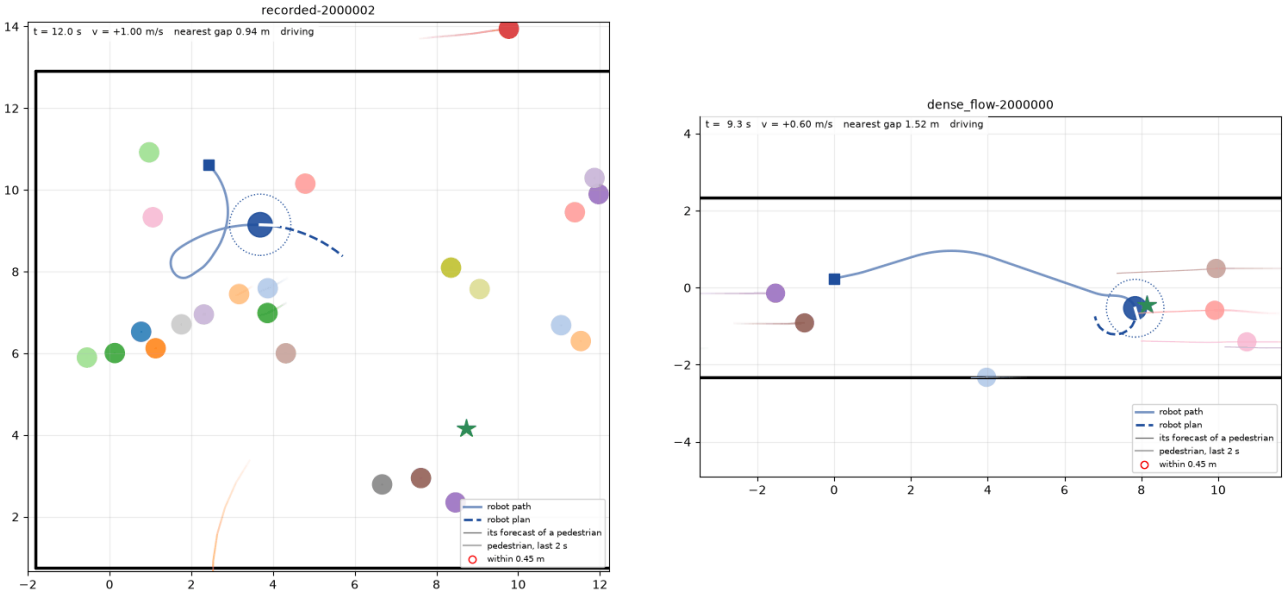


Figure 4. Frames from the v4 GIFs: a recorded STEPS scene (left) and dense flow (right). The robot's plan is dashed, its forecasts of pedestrians are thin lines, and the status line gives time, speed and nearest gap. The full GIFs are in docs/latest_policy/visuals/.

6.3 Robustness to the crowd model

The pooled forecaster is as realistic as the STEPS model in 20 s rollouts (speed 0.116 against 0.115 m/s; collision rate 0.15% against 0.17%). However, its pedestrians react about 3× less to the robot (robot_influence 0.11 against 0.28 m). Neither model has seen a robot, so the data cannot say which reaction strength is right. Objectives are therefore not comparable across crowds, and we compared behaviour metrics under a worst-case selection rule.

Policy	STEPS crowd: success / collision / < 0.45 m / recorded	Pooled crowd: success / collision / < 0.45 m / recorded
v4 (tuned in STEPS crowd)	98.5% / 0.7% / 2.6% / 100%	97.4% / 0.4% / 2.1% / 93%
Tuned in pooled crowd, seed 42	94.1% / 4.4% / 5.6% / 87%	97.8% / 0.4% / 4.1% / 87%
Tuned in pooled crowd, seed 7	94.8% / 3.7% / 6.6% / 80%	97.0% / 1.9% / 5.7% / 87%

Policies tuned in the less reactive crowd learn to cut close, and collide 5–6× more in the reactive one. The practical rule is to tune in the more responsive crowd (decision 88).

6.4 What did not beat v4 (decisions 68–92)

Attempt	Idea	Result
Cost-term changes (68–80)	Speed and retreat scales, a no-loop option, idle-turn penalty, wider bounds (v5–v7)	No variant removes dense-flow loops without losing safety. v5–v7 tie with or trail v4.
Memory (81–83)	Side commitment by turn sign; pass-side memory	No effect. Per-episode outcomes depend on batch composition, but aggregates are stable.
Progress along the whole plan (89)	Score progress over the full rollout	Not adopted.
Wait-then-go plans (90)	Two-stage candidates: hold, then drive	Untuned: no effect. Re-tuned: no gain. The first form made the robot spin.
Learned steering policy (91)	MLP 64→64→64→2 (8.4k weights) on robot-frame goal, own motion, 6 nearest pedestrians with STARS forecasts at 1 and 2 s, and the nearest obstacle. Safety shield. Behaviour cloning from v4, then OpenAI-ES (48 antithetic pairs, σ = 0.02), later population 96 with a fallback shield.	Plateaus at −0.13 to −0.20 validation, 86–90% success and 7–10% collisions; far below v4.
Contextual settings network (92)	MLP adjusts v4's parameters per step, initialised at v4	ES drifts away from v4 at every learning rate tried (0.01 down to 0.001). From an optimum of the same objective, the gradient signal is weaker than its noise.

Conclusion. v4 is close to the best this controller and objective allow in the STARS crowd. Its remaining weakness is about 0.16 collision-free loops per dense-flow episode. Further gains would need far more compute (gradient-based RL through a differentiable or model-based surrogate, or ES with thousands of members) or a different objective or crowd, for example one fitted to robot-interaction data.

7 Assumptions, judgement calls and threats to validity

7.1 Assumptions in the data and metrics

- **Timing:** 2.5 Hz, 8 observed and 12 predicted steps everywhere, inherited from the repository's STEPS preset.
- **Graph:** 16 nearest neighbours within 10 m, chosen for memory and throughput (fully connected graphs need 2.5× the memory, 140 against 57 GB). Nodes must be visible at the last observed step and seen at least twice.
- **Thresholds, picked by hand and not calibrated:** moving > 0.2 m/s, collision < 0.2 m, interacting < 2 m, implausible speed > 3 m/s. The comfort zones (0.45 m and 1.2 m) come from SEAN 2.0.
- **Best of 20** is the headline metric; it flatters every stochastic model. Joint ADE (one sample for all agents together) is reported alongside it.
- **STEPS headline numbers are low** because about 90% of people are nearly stationary. The moving-person scores are the more honest view.
- **Model selection** uses the macro mean of per-dataset validation agent ADE. ETH has no validation split, so it is tested only.
- **ATC** is capped at 80k, 10k and 50k windows by a seeded draw. Its test day (29 September) is a different recording day than training.

7.2 Simulator and navigation assumptions

- **Circularity.** In the main setup the same forecaster moves the crowd and serves as the robot's predictor. The policy may be exploiting that model's specific reactions. Mitigations: cross-crowd evaluation (6.3), and the v1 transfer across ORCA, social-force and replay pedestrians.
- **The robot is "one more person" to the crowd.** No dataset contains a robot, so how strongly people yield to it is unvalidated. The STEPS and pooled crowds differ by 3× here.

- **The objective weights are judgement calls** (decision 23). One weight was miscalibrated and then corrected: disruption 10 → 1 (decision 52). v4 was tuned on its own objective. Test scenes use disjoint seeds and recordings, but it is still the same scenario distribution.
- **2-D kinematics only.** The robot is a disc, sensing is perfect within 8 m, and there are no sensors, occlusion or map uncertainty. Pedestrians cannot pass through the robot, so all collisions are robot-caused by construction.
- **Recorded scenes** are the busiest of 8 random 30 s windows, with the robot routed through the crowd at mid-window, because random windows were too sparse. Recorded goals keep 1 m from pedestrians.
- **Simulated crowds are smoother than reality** (acceleration 0.03 against 0.12 m/s²). Long rollouts drift on average 0.67 m from recorded paths over 30 s.
- **Reproducibility.** Crowd evaluation is not bit-exact (bf16 on GPU). Differences below about 0.02 in the objective are noise.

7.3 Process decisions made autonomously

During the 30-minute loop, choices were made without you and logged for review in `docs/decisions.md` (93 entries). The most consequential:

- Using the realistic model (cvae_kl01_finetune) as the crowd instead of the most accurate one (trial 53).
- Keeping the STEPS crowd as the simulator default over the pooled crowd.
- The worst-case-across-crowds rule for adopting a new policy.
- Stopping controller cost-term work after decision 89.
- Concluding the learned-policy work after decision 92.
- Capping ATC.
- Successful multi-GPU runs exit via `os._exit` to avoid a bus error at DDP teardown.

8 Open items and suggested next steps

- **Record the ATC results** (this report's Section 4.4) as the outcome of decision 93 in docs/decisions.md and docs/results/README.md. Decide whether all7 becomes the pooled reference.
- **Robot reaction strength.** Fit or validate how much pedestrians yield to a robot, for example from robot-in-crowd data. This is the largest unknown behind every navigation number.
- **Dense-flow loops.** These would need either gradient-based RL through a surrogate or much larger ES budgets.
- **Forecaster.** A diffusion decoder, robot conditioning (node type 0 is reserved), agent entries and exits during rollouts, and an interaction-deviation metric.
- **SEAN-JAX Phases 3–4.** An Isaac Sim headless physics smoke test in Apptainer. The risks: Isaac officially supports aarch64 only on DGX Spark, and GB200 likely lacks RT cores for rendering. Also a ROS 2 Jazzy node via RoboStack (people_msgs is missing there). A JAX port only if the name should become literal.
- **Housekeeping.** Fix the stale paths in the trajectory_forecasting checkout. Bring up the GLM server if still wanted. Rotate the GitHub token.

Appendix A Where things are

Item	Path (repository-relative unless absolute)
Decision log (1–93)	`docs/decisions.md`
Results summary and figures	`docs/results/README.md, docs/results/*.png`
Latest policy page (GIFs)	`docs/latest_policy/`
Latest forecaster page	`docs/latest_forecaster/`
Forecaster design	`docs/stars_multiagent.md`
SEAN-JAX README (quick start, extension guide)	`seanjax/README.md`
Horizon how-to	`docs/horizon.md`
STEPS reference forecaster	`experiments/stars_ma/runs/cvae_kl01_finetune/checkpoints/best.pt`
Pooled forecaster (6 datasets)	`experiments/stars_ma/runs/cvae_all_lr3e-4_seed42/checkpoints/best.pt`
Pooled forecaster (7 datasets, ATC)	`experiments/stars_ma/runs/cvae_all7_lr3e-4_seed{42,1}/`
Reference navigation policy v4	`checkpoints/seanjax/cmaes_crowd_v4/best_policy.yaml`
SEAN 2.0 / RVO2 survey clones	`/scratch/09865/mmunje/.cache/sean_survey/`
ORCA reference venv and comparison script	`/scratch/09865/mmunje/.cache/orca_ref/compare.py`
Shared datasets	`/scratch/09865/mmunje/shared/datasets`

Appendix B Reproduce the headline numbers

```
python seanjax/scripts/evaluate.py --config seanjax/configs/cmaes_crowd_v4.yaml --policy
checkpoints/seanjax/cmaes_crowd_v4/best_policy.yaml --split test --device cuda
```

```
python -m baselines.evaluate_stars_ma --checkpoint
experiments/stars_ma/runs/cvae_all7_lr3e-4_seed42/checkpoints/best.pt --config
configs/stars_ma/all7.yaml --split test --output eval_test_all7.json
```

```
sbatch scripts/horizon/stars_ma.slurm --config configs/stars_ma/all7.yaml --phase forecast --run-dir
experiments/stars_ma/my_run # DDP over all GPUs of the node
```

Each run's full settings are in its config.json; more commands are in docs/stars_multiagent.md, docs/horizon.md and seanjax/README.md.